ECE 332 Lab 4 Report
Members: Shahyan, Osama, Faisal

Github Link: https://github.com/osamanalrahili-eng/Lab-4

Lab Objective

The objective of Lab 4 was to accelerate the GEMM matrix multiplication portion of the handwritten digit recognition system using the FPGA on the DE1-SoC board. In Lab 3, the CNN inference system was running mostly on the CPU, but in Lab 4 the goal was to move the main matrix multiplication operation to the FPGA using OpenCL. This lab included writing and testing a GEMM kernel, compiling it into an FPGA bitstream, integrating it with the host code, and comparing the FPGA result with the CPU result. The lab also connected the camera capture, preprocessing, CNN inference, FPGA acceleration, and digit output into one system. The lab instructions describe the overall flow as camera capture, preprocessing, GEMM on CPU or FPGA, CNN output, and display.

The Workflow

We started by working with the provided gemm_kernel.cl file and writing the OpenCL matrix multiplication kernel. The kernel had to multiply matrix A and matrix B and store the result in matrix C. Each output value depended on the correct row and column indexing, so the main focus was making sure M, K, and N were used correctly.

After editing the kernel, we tested it using the Jupyter notebook. The notebook compared our OpenCL kernel output with the CPU output, which helped us catch mistakes before running it on the board. Once the kernel output matched the CPU result, we moved to the host code and began working with the FPGA GEMM backend.

The FPGA host code was responsible for creating buffers, sending input data to the FPGA, setting kernel arguments, launching the kernel, and reading the output matrix back. This part connected the OpenCL kernel to the CNN inference system.

Prediction and Camera Testing

For the prediction part of the lab, we tested the camera input with different handwritten digits. While testing, we noticed that the ROI of the capture box included a black area, which made the digit harder for the CNN to read. Osama and Faisal worked on adjusting the camera ROI by changing the capture size and location so the image inside the red box became clearer.

We also used debug image captures to see what the CNN was actually receiving as input. Shahyan focused on testing different handwritten digits and checking the predicted output in the terminal. After repeating the tests several times, the capture and preprocessing improved, and the predictions became more consistent.

Display Output

For the display requirement, we used the LEDR display instead of the 7 segment display. We were not able to use the 7 segment display because it would require changing the hardware setup and recompiling the FPGA design. Based on the clarification from office hours, using LEDR was acceptable. The LEDR method used base address 0x1000. The predicted digit was displayed using the upper 4 bits of the red LEDs in binary format. This allowed us to show the prediction on the board without making extra hardware changes.

Timing Comparison

We compared CPU GEMM time and FPGA GEMM time to see the performance difference. The terminal output showed that the CPU and FPGA results matched, with a max difference of 0. This confirmed that the FPGA GEMM implementation was correct.

GEMM Size	CPU Time	FPGA Time	Speed Up
M=96, K=9, N=16	654 ms	29 ms	22.6x
M=1024, K=144, N=32	34 ms	53 ms	0.64x
M=256, K=288, N=64	96 ms	73 ms	1.32x

Speedup was calculated by:

Speedup = CPU Time / FPGA Time

The first layer had the best speedup. The second layer was slower on the FPGA because the FPGA time was higher than the CPU time. This can happen because using the FPGA includes overhead from transferring data, setting up buffers, and launching the kernel. The third layer had a smaller speedup, showing that performance depends on the matrix size and how much overhead is involved.

Challenges Faced

One challenge was debugging the GEMM kernel because small indexing mistakes caused incorrect output. We had to be careful with M, K, and N because mixing up the matrix dimensions would make the FPGA result different from the CPU result. The CPU and FPGA comparison helped us verify that the output was correct.

Another challenge was understanding the FPGA host code. The host code had to create buffers, transfer input data, set kernel arguments, launch the OpenCL kernel, and read the output back. If any of these steps were wrong, the FPGA backend would not work correctly.

We also had issues with the camera capture and prediction. The ROI included a black region, which made the digit harder for CNN to process. Adjusting the capture area helped make the digit clearer and improved the prediction testing.

My Contribution

Shahyan: I focused on testing the prediction part of the lab. I tried different handwritten digits and checked whether the CNN predicted the correct number in the terminal. I also helped verify that the system reached the prediction stage after the CPU and FPGA GEMM comparisons.

Osama: I focused on adjusting the camera ROI and improving the capture box so the handwritten digit was clearer for the CNN. I also helped use debug captures to check what the model was receiving as input and helped confirm that the image was better centered.

Faisal: I focused on preprocessing and testing the adjusted camera input. I helped verify that the captured digit was positioned correctly inside the ROI and checked that the predictions became more consistent after the capture area was changed.